

AIDE Speech-to-Text API — Documentation

Internal documentation for integrating with the AIDE STT service. Covers the API contract (request/response formats, errors) and how to get SSH access to the underlying VM if needed for debugging.

1. What this service does

Accepts an audio file recorded by a patient, sends it to Google Cloud Speech-to-Text (English, Australia), and returns the raw transcript as JSON. This is the first stage of AIDE's pipeline — transcript shortening/rephrasing happens in a separate downstream step, not here.

```
Patient audio → THIS SERVICE → raw transcript → (next stage: AI rephrasing)
```

2. Base URL

```
http://<VM_EXTERNAL_IP>:8000
```

To find the current external IP, run this on the VM:

```
curl ifconfig.me
```

Or check the GCP Console: **Compute Engine** → **VM instances** → **aidevm** → **External IP**.

Note: this is currently HTTP, not HTTPS. The connection — including the API key below — is not encrypted in transit yet. Don't send real patient audio through this endpoint over the public internet until HTTPS is added (reverse proxy with a TLS cert is the planned fix — flag if this becomes urgent).

3. Authentication

Every request to `/transcribe` requires an API key in the request headers:

```
X-API-Key: <the shared key>
```

Requests without a valid key get a `401 Unauthorized` response. The key itself is not written in this document — get it directly from Elijah over a secure channel (not Slack/email in plaintext, ideally a password manager or in-person). Do not commit it to any git repository, `.env.example` file, or shared doc.

The friend's frontend server (not the browser) should hold this key as an environment variable — see the integration pattern in Section 6.

4. Endpoints

GET /

Health check. No authentication required.

Request: none

Response — 200 OK :

```
{
  "status": "ok",
  "service": "AIDE Speech-to-Text API"
}
```

Use this to confirm the service is up before debugging anything else.

POST /transcribe

Transcribes an uploaded audio file.

Headers: | Header | Required | Value | |—|—|—| | X-API-Key | Yes | the shared API key | | Content-Type | Yes | multipart/form-data (set automatically by most HTTP clients/libraries when sending form data) |

Body: multipart/form-data with a single field: | Field | Type | Required | Notes | |—|—|—|—| | file | file | Yes | the audio file |

Allowed audio formats: .wav .mp3 .flac .ogg .m4a .webm

Limits: | Limit | Value | |—|—| | Max file size | 25 MB | | Max audio duration | 5 minutes | | Max simultaneous requests | 10 (additional requests queue rather than fail) |

Response — 200 OK :

```
{
  "filename": "test_audio.mp3",
  "text": "hello hi hi my name is Elijah please I need assistance here at the room 1035 my charger dropped down and my phone is about to die I need to call my parents I need help please somebody come"
}
```

Handle this response as sensitive. The `text` field is a raw, unprocessed transcript and can contain names, room numbers, and other identifying details. Do not log it, cache it, or store it outside of what AIDE's data-handling design already accounts for.

Error responses:

Status	Meaning	Example body
400	Empty or unreadable audio file	<code>{"detail": "Audio file is empty."}</code>
401	Missing or invalid API key	<code>{"detail": "Unauthorized."}</code>
	File too large (>25MB) or too	<code>{"detail": "Audio file exceeds the</code>

413	long (>5min)	25 MB limit."}
415	Unsupported file format or MIME type	{"detail": "Unsupported audio format. Allowed formats: WAV, MP3, FLAC, OGG, M4A, WEBM."}
500	Unexpected server-side failure (Google STT error, etc.)	{"detail": "Speech-to-text processing failed."}

The frontend should handle all five of these distinctly rather than showing a generic error — especially 413 / 415, since those are user-fixable (re-record shorter, or the browser sent an unexpected format).

5. Example requests

curl:

```
curl -X POST http://<VM_EXTERNAL_IP>:8000/transcribe \
-H "X-API-Key: <the shared key>" \
-F "file=@test_audio.mp3;type=audio/mpeg"
```

JavaScript (fetch , from a Node/server environment — not the browser):

```
const formData = new FormData();
formData.append("file", audioBlob, "recording.webm");

const res = await fetch("http://<VM_EXTERNAL_IP>:8000/transcribe", {
  method: "POST",
  headers: {
    "X-API-Key": process.env.AIDE_STT_API_KEY,
  },
  body: formData,
});

if (!res.ok) {
  const err = await res.json();
  // handle err.detail based on res.status
}

const { text } = await res.json();
```

6. Recommended integration pattern

Do not call this endpoint directly from browser JavaScript. Two reasons: this endpoint has no CORS headers configured, and putting the API key in client-side code exposes it to anyone who opens the browser's network tab.

Instead, route through your own backend/API route:

```
Patient's browser
| (records audio, sends to YOUR OWN backend)
▼
Your Next.js API route (server-side)
| (holds the API key, calls the VM)
▼
```

The API key lives only in your server-side environment variables (e.g. `.env.local`, or your hosting platform's environment variable settings) — never in anything that ships to the browser.

7. Getting SSH access to the VM

You generally **do not need SSH access** just to call the API — Sections 4–6 cover everything needed for frontend integration. SSH access is only useful if you need to debug the service directly (check logs, restart it, inspect the code).

Access is granted via **GCP IAM with OS Login**, not manually-shared SSH keys. This means Google manages the key exchange automatically once your Google account is granted the right roles — nothing to generate or paste yourself.

Current access status (as of this document):

- `1049viveksubramaniang@gmail.com` (Vivek) has been granted:
 - `Compute OS Login` (`roles/compute.osLogin`) — standard, non-root SSH access
 - `Compute Viewer` (`roles/compute.viewer`) — needed for `gcloud` to look up the VM's details when connecting
- OS Login is enabled on the VM (`enable-oslogin=TRUE` in custom metadata)
- Firewall rule `default-allow-ssh` (ingress, TCP:22, `0.0.0.0/0`) is confirmed active, so the connection itself isn't blocked

To connect, Vivek runs (from his own machine):

```
gcloud auth login
gcloud config set project aide-507408
gcloud compute ssh aidevm --zone=us-central1-a
```

He signs in with `1049viveksubramaniang@gmail.com` during `gcloud auth login`. The Google Cloud SDK needs to be installed first if it isn't already. On first connection, Google handles SSH key generation and exchange automatically — no manual key-pasting step.

If sudo access is needed later (installing packages, some systemctl operations), that requires upgrading his role from `Compute OS Login` to `Compute OS Admin Login` (`roles/compute.osAdminLogin`) in IAM & Admin — ask Elijah to make that change if a command fails with a permissions error.

Once connected, useful commands for debugging:

```
sudo systemctl status aide-stt      # is it running?
sudo journalctl -u aide-stt -f       # live logs
sudo systemctl restart aide-stt     # restart the service
```

Security note: `default-allow-ssh` currently allows SSH connection attempts from any IP address (`0.0.0.0/0`), not just trusted ones. OS Login still requires valid IAM permissions to actually authenticate, so this isn't an open door — but it does mean the VM is visible to automated internet-wide scanning for open port 22s. Acceptable for now; worth tightening later (restrict the rule to known IPs, or move to GCP's Identity-Aware Proxy for SSH) before this is a production system handling real patient data.

8. Known limitations (as of this version)

- **HTTP only, no HTTPS yet** — traffic including the API key is unencrypted in transit. Don't route real patient audio through this until fixed.
- **No rephrasing/shortening yet** — this endpoint returns the raw transcript only. The AI rephrasing step ("assistance requested" style output) is a separate, not-yet-built stage.
- **Single VM, no autoscaling** — concurrency is capped at 10 simultaneous transcriptions; beyond that, requests queue rather than fail, but there's no horizontal scaling yet.
- **US-hosted VM** (`us-central1-a`) — worth confirming with Noah/legal whether this affects AIDE's Australian data-residency story before real patient data flows through it.